AI ASSISTANT CODING
ASSIGNMENT-12.2

Name: M.Manideep Reddy

Enrollment no: 2303A52183

Batch: 34

Semester: VI

Branch: CSE(AIML)

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)

➢ Task: Use AI assistance to generate a Python program that implements a Stack data structure.

Instructions:

Prompt AI to create a Stack class with the following methods:

➢ push(element)

➢ pop()

➢ peek()

➢ is_empty()

➢ Ensure proper error handling for stack underflow.

➢ Ask AI to include clear docstrings for each method

```python
"""Prompt- Generate a Python program that implements a Stack using a class.
The class should contain the following methods:
- push(element)
- pop()
- peek()
- is_empty()
Use a Python list to store elements and follow the LIFO principle.
Include proper error handling for stack underflow (raise IndexError when pop() or peek() is called on an empty stack).
Add clear docstrings for the class and all methods.
Include a simple example usage in the main block. """
```

Online Python Compiler | Share | Run | Output

```python
class Stack:
    def __init__(self):
        """
        Initializes an empty stack.
        """
        self._items = []
    def push(self, element):
        """
        Adds an element to the top of the stack.
        Parameters:
        element (any): The element to be added to the stack.
        self._items.append(element). """
    def pop(self):
        """
        Removes and returns the top element of the stack.
        Returns: any: The top element of the stack.
        Raises: IndexError: If the stack is empty (stackunderflow).
        """
        if self.is_empty():
            raise IndexError("Stack Underflow Error: Cannot pop from an empty stack.")
        return self._items.pop()
    def peek(self):
        """
        Returns the top element of the stack without removing it.
        Returns: any: The top element of the stack.
        Raises: IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Stack is empty: Cannot peek.")
        return self._items[-1]
    def is_empty(self):
        """
        Checks whether the stack is empty.
        Returns: bool: True if the stack is empty, False otherwise.
        """
        return len(self._items) == 0
if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)

    print("Top element:", stack.peek())  # 30
    print("Popped element:", stack.pop())  # 30
    print("Is stack empty?", stack.is_empty())  # False
```

Status: Accepted

Memory Used: 3292 byte | Execution Time: 0.023 sec

```
Top element: 30
Popped element: 30
Is stack empty? False
```

Input

Enter your input here...

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)

➢ Task: Use AI to implement and compare Linear Search and Binary Search algorithms in Python.

Instructions:

➢ Prompt AI to generate:

➢ linear_search(arr, target)

➢ binary_search(arr, target)

➢ Include docstrings explaining:

➢ Working principle. Test both algorithms using different input sizes.

```python
"""Prompt- Generate a Python program to implement and compare Linear Search and Binary Search.

Create:
- linear_search(arr, target)
- binary_search(arr, target) (iterative, sorted array)

Include docstrings explaining working principle and time complexity. Return index if found, else -1.
Test with different input sizes and compare execution time. """
```

Online Python Compiler

Share | Run

```python
import time
def linear_search(arr, target):
    """
    Linear Search Algorithm
    Working Principle:
    Traverses the array sequentially and compares each element
    with the target value until it is found or the list ends.
    Parameters:
    arr (list): List of elements
    target (int): Element to search
    Returns:
    int: Index of the target if found, otherwise -1
    Time Complexity:
    Best Case  : O(1)
    Worst Case : O(n)
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1

def binary_search(arr, target):
    """
    Binary Search Algorithm (Iterative)
    Working Principle:
    Works on a sorted array. It repeatedly divides the search
    interval into half by comparing the target with the middle element.
    Parameters:
    arr (list): Sorted list of elements
    target (int): Element to search
    Returns:
    int: Index of the target if found, otherwise -1
    Time Complexity:
    Best Case  : O(1)
    Worst Case : O(log n)
    """
    left = 0
    right = len(arr) - 1

    while left <= right:
```

Output

Status: Accepted

Memory Used: 3604 byte | Execution Time: 0.029 sec

```
Linear Search vs Binary Search Comparison

Input Size: 10
Linear Search -> Index: 9, Time: 0.000005 seconds
Binary Search -> Index: 9, Time: 0.000005 seconds
--------------------------------------------------
Input Size: 1000
Linear Search -> Index: 999, Time: 0.000111 seconds
Binary Search -> Index: 999, Time: 0.000007 seconds
--------------------------------------------------
Input Size: 10000
Linear Search -> Index: 9999, Time: 0.001153 seconds
Binary Search -> Index: 9999, Time: 0.000010 seconds
```

Input

Enter your input here...

```python
    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

if __name__ == "__main__":

    input_sizes = [10, 1000, 10000]
    print("Linear Search vs Binary Search Comparison\n")

    for size in input_sizes:
        arr = list(range(size))
        target = size - 1
        print(f"Input Size: {size}")
        # Linear Search
        start_time = time.time()
        result_linear = linear_search(arr, target)
        end_time = time.time()
        linear_time = end_time - start_time
        print(f"Linear Search -> Index: {result_linear}, Time: {linear_time:.6f} seconds")
        # Binary Search
        start_time = time.time()
        result_binary = binary_search(arr, target)
        end_time = time.time()
        binary_time = end_time - start_time
        print(f"Binary Search -> Index: {result_binary}, Time: {binary_time:.6f} seconds")
        print("-" * 50)

    print("\nComplexity Analysis Summary:")
    print("Linear Search  : O(n)")
    print("Binary Search  : O(log n)")
    print("Binary Search is more efficient for large sorted datasets.")
```

```
Linear Search -> Index: 9, Time: 0.000005 seconds
Binary Search -> Index: 9, Time: 0.000005 seconds
--------------------------------------------------
Input Size: 1000
Linear Search -> Index: 999, Time: 0.000111 seconds
Binary Search -> Index: 999, Time: 0.000007 seconds
--------------------------------------------------
Input Size: 10000
Linear Search -> Index: 9999, Time: 0.001153 seconds
Binary Search -> Index: 9999, Time: 0.000010 seconds
--------------------------------------------------

Complexity Analysis Summary:
Linear Search  : O(n)
Binary Search  : O(log n)
Binary Search is more efficient for large sorted datasets.
```

Input

Enter your input here...

Was it helpful?

Task Description -3 (Test Driven Development – Simple Calculator Function)

➢ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

➢ Prompt AI to first generate unit test cases for addition and subtraction.

➢ Run the tests and observe failures.

➢ Ask AI to implement the calculator functions to pass all tests.

➢ Re-run the tests to confirm success.

```python
"""Prompt- Generate Python unit tests first (using unittest) for addition(a, b) and subtraction(a, b).
Include positive, negative, and zero test cases.
Then implement the calculator functions to pass all tests.
Show test execution before and after implementation. """
```

```python
import unittest

# Implementation
def addition(a, b):
    """
    Returns the sum of two numbers.
    Time Complexity: O(1)
    """
    return a + b

def subtraction(a, b):
    """
    Returns the difference of two numbers.
    Time Complexity: O(1)
    """
    return a - b

# Test Cases (Written First in TDD)
class TestCalculator(unittest.TestCase):

    # Addition Tests
    def test_add_positive(self):
        self.assertEqual(addition(5, 3), 8)
    def test_add_negative(self):
        self.assertEqual(addition(-2, -4), -6)
    def test_add_zero(self):
        self.assertEqual(addition(5, 0), 5)

    # Subtraction Tests
    def test_subtract_positive(self):
        self.assertEqual(subtraction(10, 4), 6)
    def test_subtract_negative(self):
        self.assertEqual(subtraction(-5, -3), -2)
    def test_subtract_zero(self):
        self.assertEqual(subtraction(7, 0), 7)

# Run Tests

if __name__ == "__main__":
    unittest.main()
```

```
Status: Accepted
Memory Used: 9248 byte | Execution Time: 0.146 sec

......
----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
```

Input

Enter your input here...

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

Task: Use AI assistance to generate a Python program that implements a Queue data structure.

Instructions:

➢ Prompt AI to create a Queue class with the following methods:

- enqueue(element)
- dequeue()
- front()
- is_empty()

➢ Handle queue overflow and underflow conditions.

➢ Include appropriate docstrings for all methods.

```python
"""Prompt- Generate a Python program that implements a Queue class.

Requirements:
- Methods: enqueue(element), dequeue(), front(), is_empty()
- Use a list for storage.
- Handle queue overflow (fixed size) and underflow using exceptions.
- Add proper docstrings for the class and all methods.
- Include example usage in main. """
```

Online Python Compiler Share Run

```python
class Queue:
    def __init__(self, capacity):
        """
        Initializes the queue with a fixed capacity.
        Parameters: capacity (int): Maximum number of elements the queue can hold.
        """
        self.capacity = capacity
        self.queue = []

    def enqueue(self, element):
        """
        Adds an element to the rear of the queue.
        Parameters: element (any): Element to be added.
        Raises: OverflowError: If the queue is full.
        """
        if len(self.queue) >= self.capacity:
            raise OverflowError("Queue Overflow: Cannot enqueue to a full queue.")
        self.queue.append(element)
    def dequeue(self):
        """
        Removes and returns the front element of the queue.
        Returns: any: The removed front element.
        Raises: IndexError: If the queue is empty (underflow).
        """
        if self.is_empty():
            raise IndexError("Queue Underflow: Cannot dequeue from an empty queue.")
        return self.queue.pop(0)

    def front(self):
        """
```

Output

Status: Accepted

Memory Used: 3324 byte | Execution Time: 0.022 sec

```
Enqueueing elements...
Front element: 10
Dequeued element: 10
Front after dequeue: 20
Is queue empty? False
```

Input

Enter your input here...

```python
class Queue:
    def dequeue(self):
        if self.is_empty():
        return self.queue.pop(0)

    def front(self):
        """
        Returns the front element without removing it.
        Returns: any: Front element of the queue.
        Raises: IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Queue is empty: No front element.")
        return self.queue[0]

    def is_empty(self):
        """
        Checks whether the queue is empty.
        Returns: bool: True if empty, otherwise False.
        """
        return len(self.queue) == 0

if __name__ == "__main__":
    q = Queue(3)
    print("Enqueueing elements...")
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)

    print("Front element:", q.front())
    print("Dequeued element:", q.dequeue())
    print("Front after dequeue:", q.front())
    print("Is queue empty?", q.is_empty())
```

```
Status: Accepted
Memory Used: 3324 byte | Execution Time: 0.022 sec

Enqueueing elements...
Front element: 10
Dequeued element: 10
Front after dequeue: 20
Is queue empty? False
```

Input

Enter your input here...

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)

➢ Task:

Use AI to implement Bubble Sort and Selection Sort algorithms and compare their behavior.

Instructions:

➢ Prompt AI to generate:

• bubble_sort(arr)

• selection_sort(arr)

➢ Include comments explaining each step.

➢ Add docstrings mentioning time and space complexity.

```python
"""Prompt- Generate Python code to implement Bubble Sort and Selection Sort.

Create:
- bubble_sort(arr)
- selection_sort(arr)

Include:
- Clear comments explaining each step
- Docstrings mentioning working principle
- Time and space complexity analysis
- Example usage to test both algorithms """
```

```python
def bubble_sort(arr):
    """
    Bubble Sort Algorithm
    Working Principle:
    Repeatedly compares adjacent elements and swaps them
    if they are in the wrong order. Largest elements "bubble up"
    to the end in each pass.

    Time Complexity:
    Best Case: O(n)
    Worst Case: O(n^2)
    Average Case: O(n^2)

    Space Complexity:
    O(1) (In-place sorting) """
    n = len(arr)

    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

    return arr
def selection_sort(arr):
    """
    Selection Sort Algorithm
    Working Principle:
    Repeatedly finds the minimum element from the unsorted
```

Status: Accepted

Memory Used: 3220 byte | Execution Time: 0.022 sec

```
Original List: [64, 34, 25, 12, 22, 11, 90]

Bubble Sort Result:
[11, 12, 22, 25, 34, 64, 90]

Selection Sort Result:
[11, 12, 22, 25, 34, 64, 90]
```

Input

Enter your input here...


```python
def selection_sort(arr):
    """
    Selection Sort Algorithm
    Working Principle:
    Repeatedly finds the minimum element from the unsorted
    part and places it at the beginning.

    Time Complexity:
    Best Case: O(n^2)
    Worst Case: O(n^2)
    Average Case: O(n^2)

    Space Complexity:
    O(1) (In-place sorting) """
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr
# Testing

if __name__ == "__main__":
    original_list = [64, 34, 25, 12, 22, 11, 90]
    print("Original List:", original_list)
    print("\nBubble Sort Result:")
    print(bubble_sort(original_list.copy()))
    print("\nSelection Sort Result:")
    print(selection_sort(original_list.copy()))
```

Status: Accepted

Memory Used: 3220 byte | Execution Time: 0.022 sec

```
Original List: [64, 34, 25, 12, 22, 11, 90]

Bubble Sort Result:
[11, 12, 22, 25, 34, 64, 90]

Selection Sort Result:
[11, 12, 22, 25, 34, 64, 90]
```

Input

Enter your input here...

Was it helpful?